

APK Sentinel: complete project & pipeline guide

A self-contained explanation of the whole system, written for a CS undergraduate. No prior knowledge of Android internals, malware analysis, or machine learning is assumed. Read top to bottom; each section builds on the last.

Built for the **PSB Cybersecurity, Fraud & AI Hackathon 2026** (Bank of India · IIT Hyderabad · DFS, Ministry of Finance · IBA). Target threat: fraudulent Android apps impersonating Indian banks, UPI apps, and government services.

PART 0 · THE VOCABULARY YOU NEED FIRST

0.1 · What is an APK?

An **APK** (“Android Package Kit”) is the file you install to put an app on an Android phone. It is not a special binary format — it is literally a **ZIP archive** with a fixed internal layout and a `.apk` extension. Rename `whatsapp.apk` to `whatsapp.zip`, open it, and you will see its contents.

MEMBER	WHAT IT IS
<code>AndroidManifest.xml</code>	The app's “ID card”. Declares the package name (e.g. <code>com.whatsapp</code>), the app label (the name under the icon), the permissions it wants (SMS, camera, ...), and its components (screens, background services, broadcast receivers). Stored as <i>binary</i> XML, not text.
<code>classes.dex</code>	(sometimes <code>classes2.dex</code> , ...) The compiled program. Java/Kotlin source is compiled to Dalvik bytecode and packed into a <code>.dex</code> (Dalvik Executable) file. This is <i>one big file containing the entire app plus every library it bundles</i> .
<code>resources.arsc</code>	A compiled table of the app's strings, colours, layouts, and so on.
<code>res/</code>	Images, icons, raw resource files.
<code>lib/</code>	Optional native code — <code>.so</code> shared libraries (compiled C/C++ for arm64, x86, ...). Only about 12% of apps have these.
<code>META-INF/</code>	The digital signature . Every APK is cryptographically signed by whoever built it.

Two facts drive the whole project:

1. The **.dex** is where the app's real behaviour lives, but it is compiled bytecode, not readable text. To read it we must *decompile* it back to something Java-like.
2. The **manifest** is the app's self-declared identity. A fake bank app puts “YONO SBI” as its label and asks for SMS permissions here — visible without running anything.

0.2 · Key security terms

Static analysis	Examining an app <i>without running it</i> : unzip it, read the manifest, decompile the .dex, scan the code for suspicious patterns. Safe — you never execute malware — but blind to anything the app only does at runtime.
Dynamic analysis	<i>Running</i> the app in a controlled, disposable environment (an emulator) and watching what it actually does: what servers it contacts, what APIs it calls. Sees real behaviour but is expensive and can be evaded.
Obfuscation	Deliberately making code hard to read: renaming everything to a, b, zz; encoding strings (e.g. base64) so a scanner can't find the C2 address as plain text. Malware does this to hide.
C2 (command-and-control)	The attacker's server. Malware “phones home” to it to receive commands (“forward this OTP”, “show this fake screen”) and to upload stolen data.
IOC (indicator of compromise)	A concrete, blockable artefact: a domain, an IP, a URL, a bitcoin address. What a bank's security team would load into a firewall.
SHA-256	A cryptographic hash : a function turning any file into a fixed 64-hex-character fingerprint. One-way (you cannot reverse it back to the file) and collision-resistant (two different files essentially never share one). Change one byte and the fingerprint changes completely. Note: a hash is not encryption — nothing is hidden or recoverable; it is an identity fingerprint.
YARA	A pattern-matching language for malware. You write “rules” that say “if this file contains these strings/bytes in this combination, flag it.” Think of it as regex designed for malware triage.

0.3 · What is F-Droid, and why do we care?

F-Droid is an app store that only ships **free and open-source** Android apps. Because every app on it is open-source and community-vetted, it is a convenient, legal, large source of apps we are confident are **benign**. We use F-Droid apps as our “clean” control group — the *negative* examples a detector must **not** flag. It has one important weakness we return to repeatedly: it contains **zero real banking apps**, which limits what our benign set can prove.

1.1 · The threat

In India, banking increasingly runs through phone apps and **UPI** (Unified Payments Interface — the instant bank-to-bank system behind Google Pay, PhonePe, BHIM, Paytm). Money moves with a 4–6 digit **OTP** (one-time password) or **UPI PIN**. Attackers build fake Android apps that:

- **impersonate** a real bank / UPI app / government app (same name, same icon),
- get installed **outside the Play Store** — sent over WhatsApp or SMS as “SBI Reward.apk”, “Aarogya Setu update”, “Income Tax refund”,
- then **steal the OTP** — by reading incoming SMS, by drawing a fake login screen over the real app, or by abusing accessibility features — and drain the account.

This is **fraud**, not just a software bug. The victim installed it themselves, believing it was their bank.

1.2 · Why it is hard

- The malicious app *looks* identical to the real one.
- The dangerous logic is often **obfuscated** or downloaded **after** install, so the app looks clean when first scanned.
- Generic antivirus asks “is this app insecure?” — the wrong question. The right question is “**is this app pretending to be *your bank*, and can we prove it?**”

That last sentence is the entire thesis of this project.

1.3 · Past cases: real malware families this project is built against

These are documented, real-world Android banking-trojan families. Our detection logic and knowledge base are grounded in their actual behaviour.

FAMILY	SIGNATURE BEHAVIOUR
Cerberus / Alien	Overlay attacks + accessibility abuse + SMS theft; leaked source spawned many clones.
Anubis	SMS interception, keylogging, ransomware module, downloads its payload after install (dropper).
FluBot	Spread by smishing (SMS with a link); read contacts and self-propagated; used DNS-over-HTTPS to hide C2.
SharkBot / Vultur	Automated Transfer System (ATS): drives the victim's <i>real</i> banking app via accessibility; Vultur adds hidden VNC screen control.
TeaBot / Anatsa	Staged droppers that passed Play Store review as clean “utilities”, then fetched the banking payload.
SOVA / Ermac / Octo / Hydra	Overlay kits with hardcoded lists of targeted bank packages.
XBot (org.merry.core)	In our own corpus. Beacons its C2 the moment it launches. We captured this live — see L2.
India-specific clones	Fake “SBI Quick Support”, four separate fake “Aarogya Setu” apps, fake “ICICI Rewards”, an “iMobile”/ICICI namespace squat.

1.4 · Common characteristics of Android banking malware

Across families the same handful of **capabilities** recur — and these are exactly what our detectors look for.

1. **SMS interception** — the “SMS trifecta” of RECEIVE_SMS + READ_SMS + SEND_SMS, a combination almost no legitimate app needs. Used to steal OTPs and forward them to the attacker.
2. **Overlay attacks** — drawing a fake window (TYPE_APPLICATION_OVERLAY) on top of the real bank app to capture what you type.
3. **Accessibility abuse** — Android's Accessibility service exists for screen readers; malware abuses it to read the screen and auto-click, even completing transfers by itself (ATS).
4. **Notification / clipboard theft** — reading OTPs from notifications, or swapping a copied UPI ID for the attacker's.
5. **Impersonation** — package, label, and icon copied from a real brand, but signed by an unknown certificate.
6. **C2 communication** — often obfuscated (base64- or XOR-encoded server address).

7. **Persistence & droppers** — device-admin to resist uninstall; downloading a second-stage payload; restarting on boot.
8. **Evasion** — detecting emulators and sandboxes, and staying quiet when watched.

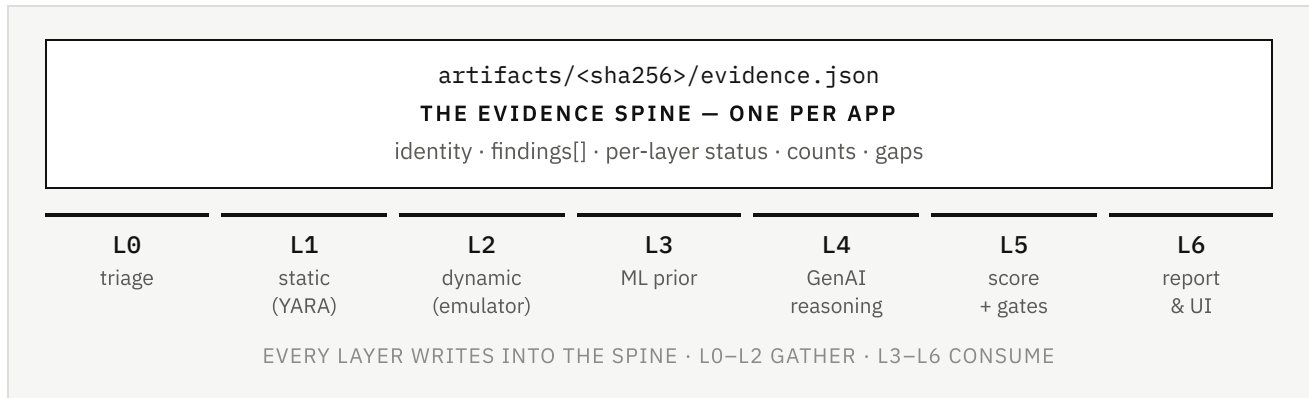
PART 2 · COMPETITORS, AND WHERE WE ARE DIFFERENT

TOOL / APPROACH	WHAT IT DOES	WHERE IT FALLS SHORT — AND WE SHINE
MobSF	The standard open-source mobile app scanner. Great at “is this app <i>insecure</i> ?” — hardcoded secrets, weak crypto, exported components.	Answers a developer's question, not a fraud analyst's. It does not ask “is this impersonating a <i>specific</i> Indian bank?” No brand or impersonation reasoning, no India-targeting logic.
VirusTotal	Aggregates around 70 antivirus verdicts by file hash.	A black box: it says “35/70 engines flag this” but cannot show you why in evidence you can audit. Useless on a brand-new sample no engine has seen.
Commercial AV / EDR	Signature plus heuristic detection.	Verdict-only (“malicious”), no per-finding evidence trail; not tuned to Indian BFSI (banking / financial services / insurance) impersonation; closed.
Pure ML classifiers (academic)	Train a model on features, output a probability.	A single opaque number. No explanation a bank's compliance team could defend, and prone to learning shortcuts (“uses a modern library → benign”) instead of real malice.

OUR DIFFERENTIATORS

1. **Impersonation-first.** The headline question is brand impersonation of Indian banks, UPI, and government apps, backed by evidence, not generic insecurity.
2. **Every number is auditable.** The final score is not a black box — each point traces to a named signal and the exact evidence ID that produced it (see L5). A judge, or a bank auditor, can click from “score 88” down to “because rule X fired on file Y at line Z.”
3. **The AI is not allowed to make things up.** Where we use a large language model (L4), every claim it makes that a computer can check is **mechanically re-checked** before it is kept. The model explains; it never *asserts facts*.
4. **Honest about its own limits.** The system *refuses to overclaim*. When its statistical support is too weak it stamps its own score unsupported rather than pretending to be sure (see L5, the T24 problem). This intellectual honesty is itself a design feature.

The pipeline has **seven layers**, L0 → L6. They are glued together by one shared file per app called the **evidence spine**.



- **L0–L2 gather evidence** — they look at the app.
- **L3–L6 consume evidence** — they reason over what L0–L2 found.
- The **spine** is the single source of truth. Every finding in it has a stable **ID** (F001, F002, ...) so later layers can *cite* evidence rather than re-assert it.

Why a shared spine and not just passing data down a chain? Because layers run at different times, some optional, some failing. A shared, append-only record lets any layer merge its part without clobbering another's, lets us tell “never ran” apart from “ran and found nothing,” and gives every downstream claim a stable ID to point at. This is implemented in `spine.py`, whose one golden rule is: **`update_layer()` is the only function that writes the spine** — it merges *only its own layer's* block, and writes atomically (write-to-temp then rename) so a crash never leaves a half-written file.

Code: `L0/ingest.py` · `L0/certinfo.py` · `L0/impersonation.py` · `L0/promote.py` · `L0/bank_whitelist.json`

Job: open the APK, extract its identity, and answer the one question that makes this project different — *is this app impersonating a known Indian bank?*

What L0 does, step by step

1. **Hashing.** Compute MD5 and **SHA-256** of the file (`compute_hashes`).
2. **Manifest harvest.** Read package name, label, version, and permissions; flag the “high-risk” permissions (SMS, accessibility, overlay, install, device-admin, ...).
3. **Icon perceptual hash.** Extract the launcher icon and compute a **pHash** — a hash that is *similar for visually similar* images, unlike SHA-256, which changes completely for a one-pixel edit. This lets us detect an icon that *looks like* a bank's logo even if the file isn't byte-identical.
4. **Certificate parsing** (`certinfo.py`). Read the signing certificate and classify **signer anomalies**: signed with a debug key, the public AOSP test key, an empty or placeholder identity, an absurd validity period, or a self-signed cert claiming to be a major vendor.
5. **Impersonation check** (`impersonation.py`) — the core.
6. **Track routing.** Decide what L1 must do: jadx (Dalvik only), ghidra (native only), or both.
7. **Promote** the impersonation and certificate findings into the shared spine with evidence IDs (`promote.py`).

The impersonation engine — mechanism and reasoning

We keep a hand-curated **whitelist of 40 banks, UPI apps, and government entities** (`bank_whitelist.json`). Each entry has the official package name, the real app label and alternate labels, and **curated “brand tokens”** — for State Bank of India: strong tokens `sbi`, `yono`; phrase `state bank of india`; official package `com.sbi.lotusintouch`; verified vendor prefix `com.sbi`.

Matching rules — **whole-token and whole-phrase only**:

- “SBI Quick Support” splits into `{sbi, quick, support}` → hits the strong token `sbi` → **brand claim on SBI**.
- The app's package `direct.uujgiq.imobile` contains the segment `imobile` (ICICI's brand) but isn't ICICI's official package → **namespace squat**.

- If the app claims a brand **but isn't the official package**, that is a **brand impersonation** finding. Severity escalates to **critical** if a signer anomaly is also present — “claims to be SBI *and* is signed with a debug key.”

Why these exact rules? Because the naive versions failed, measurably:

- **No fuzzy matching.** An early version used string-similarity and scored the benign note app “duckAssist” against the Income Tax alt-label “iAssist” at 0.706 — raising a **critical impersonation finding on an innocent notepad**. Whole-token matching fixes it: duckAssist → {duck, assist}, which cannot hit iassist.
- **No substring matching.** “BOI Mobile” compacts to boimobile, which *contains* imobile (ICICI) — a cross-brand false positive manufactured out of thin air. Banned.
- **Manifest identity only — never the code.** L0 looks *only* at the package name and label, never at strings inside the .dex. This is deliberate and load-bearing: an honest expense-tracker (PennyWise) legitimately mentions bank names in its text; if we scanned its code we would falsely flag it. Brand references *in code* are L1's job, not L0's.
- **self_signed is ignored.** Every Android APK is self-signed (verified 22 of 22 locally, benign included). A signal present on 100% of everything discriminates nothing, so it is stripped everywhere. Only signer *anomalies* count.

Why SHA-256 matters here, beyond “it's a filename”

The SHA-256 is the **primary key of the entire evidence model**, and it earns its keep in several concrete ways:

- **The join key across all seven layers.** Everything about one app lives at artifacts/<sha256>/evidence.json. That is how L5 knows which L1 findings go with which L0 identity — they share a hash.
- **Content-addressing → automatic dedup.** Two copies of the same malware under different filenames (SBI.apk, reward.apk) produce the **same** hash, so we analyse them once, not twice.
- **Tamper-evidence.** Change one byte and the hash changes — so an evidence record can never be silently attached to a modified file.
- **Idempotent re-runs.** Re-ingesting identical bytes lands on the same spine; if nothing changed, update_layer is a no-op. “Did anything change?” is answerable by a timestamp, not a diff.
- **Reputation lookups.** Optional VirusTotal and MalwareBazaar queries are keyed **by the file's SHA-256** — the natural way to ask “has anyone seen this exact file before?”

The idea of a *growing* library of malware data

L0 keeps a local **threat cache** (`threat_cache.json`), keyed by SHA-256. When a hash is known we can attach prior knowledge instantly and offline — no network needed, important for a bank that will not send samples to the cloud. The vision is a library that grows: every confirmed sample's hash and verdict recorded so the next encounter is instant. *Status note: today this cache is seeded manually, not auto-appended during runs — a designed capability, not yet a live learning loop.*

L0 FAILURE POINTS

- **ResParserError** — roughly 3.8% of malware ships a deliberately malformed resource table to crash parsers, an anti-analysis trick. L0 records the failure rather than dying silently.
- **Icon extraction** used to fail on about half of modern apps because the default picks an “adaptive icon” stored as binary XML that image libraries cannot decode. Fixed by walking a DPI ladder to find a real bitmap.
- **The whitelist is only as good as its curation.** Auto-deriving brand tokens was banned because it proposed dangerous ones (assist, phone, and even Devanagari बैंक for *every* bank). Human curation is mandatory.

L1 · STATIC ANALYSIS

Code: `L1/l1.py` · `L1/engines/yara_scan.py` · `L1/schema.py` · `L1/yara_templates/*.yar`

Job: without running the app, decompile it and scan its code for the malware behaviours listed in Part 1.4.

What “static analysis” means, with a simple example

Imagine you are handed a program and told “don't run it — is it dangerous?” You would read the source looking for tell-tale lines. Suppose you find, in one class:

```
SmsMessage msg = SmsMessage.createFromPdu(pdu);           // reads an incoming SMS
String body = msg.getMessageBody();                       // extracts its text
smsManager.sendTextMessage("+8613...", null, body, ...); // forwards it abroad
```

You do not need to run it to know: **this app reads your incoming SMS and forwards it to a foreign number** — the exact OTP-theft mechanism. That is static analysis. L1 automates exactly this kind of reading, at scale, with YARA rules instead of human eyes.

The tech stack

- **jadx** — a decompiler. It takes the compiled `classes.dex` (Dalvik bytecode) and reconstructs readable **Java source** (`jadx_src/`). We bundle jadx and its own JDK 17 in `tools/` so nothing depends on the system Java.
- **YARA** — the pattern-matching engine that scans that source, the raw APK, and the dex for malicious patterns.
- **androguard** — a Python library that parses APKs and dex files, used to read the manifest and to split the dex into individual classes.
- **Ghidra** (*optional, currently absent*) — for decompiling native `.so` libraries. Only about 12% of samples have native code, so the pipeline degrades gracefully without it.

L0 decides the “track”; L1 dispatches to the matching engine (`jadx_analyze`, `ghidra_analyze`, or `combo_analyze`) and never re-derives the routing.

How YARA works, and the three-pass design

A YARA rule looks like this, simplified:

```
rule Android_SMS_Intercept {
  meta:
    category = "sms_intercept"
    severity = "high"
  strings:
    $a = "createFromPdu"
    $b = "getMessageBody"
    $c = "sendTextMessage"
  condition:
    2 of them          // fire if at least 2 of the 3 strings appear
}
```

The **condition** is the unit of detection. This is subtle, and the source of the hardest-won lessons in the project.

- **Pass 1 — the container.** Scan the raw APK (the ZIP itself) with rules that reason about **ZIP structure** (central directory, META-INF layout). Only these `scope="apk"` rules run here.
- **Pass 2 — the members, split per class.** The `.dex` holds the app's code, but it is *the whole app concatenated* — your code plus every bundled library. If you scan the whole dex as one buffer, a condition like “SMS strings AND network strings” can be satisfied by *unrelated*

pieces of library code sitting far apart, producing a false positive. So L1 **splits the dex into one buffer per class** (using androguard) and scans each class separately. A class is the unit where “these two behaviours are co-located” actually means something.

- **Pass 3 — the decompiled Java source, one file at a time.** Never concatenated, for the same reason.

Why this obsessive separation? Measured failures forced it:

- Rules were originally gated on `uint32(0) == 0x504B0304` (the ZIP magic “PK”). YARA's `uint32` is **little-endian**, so this comparison is *never true* — 27 of 43 rules were silently dead. Fixed to `uint32be`.
- A `classes.dex` starts with `dex\035`, not “PK”, so **behaviour rules gated on ZIP magic could never match the one place the app's strings actually live**. This one bug hid roughly 92% of detections. Removing the gate on the member pass took malware-category detection from 8% → 37% of the corpus.
- Whole-dex scanning made an *expense tracker* match both an OTP-stealer rule and a ransomware rule at once. Per-class scanning fixed it.
- Batching source files together caused a **100% false-positive rate on benign apps**, because conditions were satisfied by strings scattered across unrelated files.
- A dex stores API calls as *bytecode operands*, not string literals. Scanning only string constants found 0 of 4 SMS APIs in a confirmed SMS trojan; adding the `invoke` and `field` operands found all four.

Every finding is one **L1Finding** (`schema.py`) with a category (`sms_intercept`, `overlay_attack`, ...), severity, evidence text, location, and MITRE ATT&CK technique IDs. One finding per (`rule`, `sample`) — breadth (how many files or strings matched) is recorded as detail, not as duplicate findings.

Versioning and IOCs

- `ruleset_version()` hashes the rule files **plus** a scanner-behaviour version number. Why the extra number? Because we once changed *how the scanner works* without editing any `.yar` file — and the corpus runner, which skips already-done samples by `ruleset_version`, would have silently reused stale results. Hashing the behaviour version too forces a re-scan.
- **IOCs** (domains, IPs, URLs) are extracted but deliberately kept **separate from findings**. An IOC is not a detection — a benign app contacting a domain is normal. IOCs are what L6 exports *after* a verdict; they never inflate the score.

L1 FAILURE POINTS

- **About 40% of samples don't fully decompile within the 180-second jadx timeout** (decompilation_partial). “The rule missed it” and “jadx never produced the file” are different facts, and the spine records the coverage gap so we don't confuse them.
- **18 of about 51 rules still fire on nothing**, including both ransomware rules. They aren't broken — each matches a buffer of its own strings when tested alone — the corpus just doesn't contain their exact vocabulary co-located in one class. Detection currently comes from L0 impersonation plus a set of measured “BFSI primitive” rules, not from every legacy rule.
- **jadx output is the disk hog** — roughly 112 MB of decompiled source per 2 MB APK. The corpus runner deletes it per sample after findings are safe.

L2 · DYNAMIC ANALYSIS (DETONATION)

Code: L2/sandbox/orchestrator.py · safety.py · mitm_addon.py · honeypot.py · frida_scripts/
· L2/promote.py

Job: actually **run** the malware in a disposable, network-isolated Android emulator and record what it does — the behaviour static analysis cannot see.

Why an emulator, and why *this* one

Running live malware on anything you care about is obviously reckless. We run it in an **Android emulator** — a full virtual phone — that we can wipe after each run. Specifically the AVD (“Android Virtual Device”) named **sentinel30**: Android 30, Google-APIs x86_64 image, hardware-accelerated via KVM.

Why sentinel30 specifically and not “just the Android SDK emulator”? It *is* an SDK emulator — but a *specifically configured, named, reproducible* one. The generic default AVD (sentinel) **core-dumped on boot** on our hardware and ate days of debugging. sentinel30 is the configuration that actually boots reliably (15–20 seconds), lets us get root, run guest iptables, and — critically — detonates real malware. The lesson: “use the emulator” is not a plan; a *specific, verified, reproducible* emulator config is.

The safety layer — network isolation, fail-closed

Before *any* malware runs, safety.py locks the emulator's network down with iptables rules **inside the guest** (enforce_isolation):

- Allow only **loopback** (127.0.0.1) and the **host subnet** (10.0.2.0/24, which reaches our proxy, DNS relay, and adb).
- **DNAT** (redirect) all outbound port 80/443 traffic to our proxy.
- **DROP everything else.**

Then `verify_isolation()` **proves** it worked, fail-closed, with two checks:

- **Negative:** a ping to a real internet host (8.8.8.8) **must fail**.
- **Positive:** the proxy **must** be reachable — so a “block *everything* including the proxy” misconfiguration cannot masquerade as success.

Only if both pass does detonation proceed. This is the difference between “we think it's isolated” and “we verified the malware cannot reach the real internet.”

Frida — watching from inside the app

Frida is a *dynamic instrumentation* toolkit. It injects into a running process and lets you **hook** functions — intercept a call, read its arguments, even change them, at runtime. We use it to plant hooks on the exact APIs banking malware abuses: `sendTextMessage` (outgoing SMS), SMS-receiver paths, crypto calls. When the malware calls one, Frida reports it. This is how we see “the app just tried to send your OTP to +86...” even if the code was obfuscated — because at runtime the real call must happen.

We also **SSL-unpin** with a Frida script (`ssl_unpin.js`) so even HTTPS traffic can be inspected by the proxy: the app thinks it is talking securely; we can still read it.

mitmproxy and the honeypot — containing and faking C2

mitmproxy (“man-in-the-middle proxy”) sits between the app and the network. All the app's web traffic is redirected through it via the DNAT rule above. Our addon (`mitm_addon.py`) does two things:

1. **Contain.** Every request to an unknown host is **blocked by default** — answered locally with an empty `200 {}`, tagged `blocked: true`, and **never forwarded** to the real internet. So when XBot POSTed to its real C2 at 192.227.137.154, that request was *blocked*, not leaked out.
2. **Honeypot / fake responses.** For *known* banking-API shapes, the addon returns **convincing fake data** (`honeypot.py`) — fake UPI transactions, fake account balances, fake beneficiary lists with IFSC codes. The point: some malware only reveals its next move if it *believes* it reached a live server. Feeding it a plausible fake reply can coax out deeper behaviour.

There is a real **tension** here, and we are honest about it: *blocking* the C2 keeps us safe but means the server never sends its next command, so later stages — the actual SMS theft, the overlay — may not fire. *Observing* everything means faking a plausible C2 reply. The honeypot is our attempt to have both; it is not perfect.

DroidBot and the GenAI navigator — driving the UI

Malware often waits for user interaction — tap a button, submit a form. We use **DroidBot** to automatically explore the app's screens and click through them. The planned **GenAI navigator** (`genai_navigator.py`, reusing L4's model) is smarter: it looks at the current screen and decides *where a human victim would tap* to reach the malicious flow — “fill the login form and press Submit.”

- **Benefit:** it can reach payload code hidden behind a specific UI path (a login submit handler) that random clicking would miss.
- **Caveat:** it costs model calls, can be wrong, and — like all of L4 — must never be trusted to *invent* a finding. It only drives the UI; the *evidence* still comes from Frida, mitmproxy, and tcpdump watching what actually happens.

THE PROOF IT WORKS

On 2026-08-26, running the XBot trojan through this exact pipeline produced our **first live malicious capture**: on launch — it has a `BOOT_COMPLETED` receiver, so no UI was needed — XBot beacons its C2 with `POST http://192.227.137.154/request.php` and a base64 body that decoded to a “fetch second-stage script” command. Every prior run had been attach-only with zero findings. This is dynamic analysis doing its job.

How L2 joins the spine

`L2/promote.py` translates observed behaviours into spine findings, keeping the crucial distinction: `observation = "observed"` (it *actually happened* at runtime) versus L1's "inferred" (a static rule matched). And it distinguishes “**ran and saw nothing**” (**complete**) from “**never ran**” (**skipped**) — collapsing those would let a failed detonation read as a clean app.

L2 FAILURE POINTS

- **Incoming-SMS hook missing** — the Frida pack hooks *outgoing* `sendTextMessage`, but XBot-style OTP theft happens on the *incoming* SMS path (broadcast receiver / `createFromPdu`). A known gap.
- **Payloads gated behind a form submit** don't fire without the right UI navigation — what the GenAI navigator targets.
- **Evasion**: sophisticated malware detects the emulator and stays quiet.

L3 · MACHINE-LEARNING PRIOR

Code: `L3/unified_features.py` · `L3/unified_train.py` · `L3/unified_predict.py`
· `L3/model/unified_lgbm.joblib`

Job: give a *bounded, calibrated* second opinion — “how much does this app statistically resemble known malware?” — that nudges the score by at most ± 10 points. It is a **prior**, never a verdict.

The dataset

- **Malware:** CICMalDroid banking-labelled samples plus our GitHub malware corpus.
- **Benign:** F-Droid apps.
- After near-duplicate removal (below): **about 1,166 clusters** from 3,317 samples, with **7,834 features**.

Which “columns” (features) we chose, and why

A naïve feature extractor emits one token per method in the app — around 100,000 tokens for a modern app, roughly 99% of them `androidx`, `kotlin`, or Google **library** signatures. If you feed those in, the model learns a **shortcut**: “uses modern `androidx` → benign” — because our benign apps (F-Droid) happen to use those libraries and our malware doesn't. That is not detecting malice; it is detecting *which dataset the file came from*.

So `unified_features.py` keeps only **meaningful, security-relevant tokens**:

- **Manifest families** that are cross-app by nature: requested permissions, intent-filter actions, hardware features, URL domains.
- **API calls only into security-relevant framework classes**: telephony/SMS, accessibility, device-admin, package-installer, notifications, clipboard, screen capture, crypto, reflection, dynamic code loading (`DexClassLoader`), and so on. A fixed allow-list of *classes*, identical for

every app — capability selection, not per-sample tuning.

- **Synthetic capability aggregates** (`cap:sms`, `cap:accessibility`, `cap:overlay`, ...). Each fires if *any* of several tokens for that capability is present. This is robust: it survives an obfuscator renaming one method, and its **absence** is itself a first-class signal — the analysis showed SMS capability is present in about 80% of malware versus 11% of benign, so *absence* of a capability discriminates strongly.

Data cleaning and processing

1. **Meaningful-token filter** (above) — kills the library-fingerprint shortcut.
2. **Document-frequency floor and ceiling**: drop tokens present in only one app (memorised sample IDs → overfitting) and tokens present in nearly every app (no discrimination — e.g. `AccessibilityService`, which is at roughly a 100% base rate).
3. **Near-duplicate deduplication before evaluation**: malware corpora are full of trivial variants of the same family. We cluster at cosine ≥ 0.97 and keep one per cluster — **2,151 of 3,317 samples removed (65%)**. Without this, the same malware appears in both train and test and the score is inflated.
4. **Frozen vocabulary**. The `{token → column}` map is fixed at training time and persisted (`unified_vocab.json`); prediction uses the *exact same* map, so there is **no train/serve skew** — the same feature always means the same column. This fixed a real bug in the older LAMDA-based model, which reproduced only 0.4–1.8% of its training vocabulary at serve time.

The model choice

LightGBM — a gradient-boosted decision-tree classifier. Why:

- Excellent on **sparse, high-dimensional binary features** — exactly what a bag-of-security-tokens is.
- Fast to train and predict; runs on CPU; no GPU needed, which is on-prem friendly.
- Naturally handles feature interactions (trees) without manual engineering.

We add **isotonic calibration** on a held-out slice so the output is a *probability you can trust* — a “0.9” really means about 90% — rather than an uncalibrated tree score. Evaluation uses a **group-disjoint split**: whole malware families fall entirely into train *or* test, so a variant of a family cannot leak across the split.

The honesty caveat — important, and stated in the code

Held-out CV AUROC is **0.9963** — which we flag as a **warning, not a triumph**. Because benign = F-Droid and malware = CICMalDroid, the *source* is almost perfectly correlated with the *label*. Any AUROC ≥ 0.99 is treated as **source-artifact leakage — an upper bound, not real skill**. A trustworthy banking false-positive rate needs *benign banking apps* in the corpus, and F-Droid has **zero**. So L3 stays a **generic ± 10 prior**, explicitly forbidden from making a banking-specific claim.

L3b is a *separate* banking-specific model, trained on CICMalDroid banking plus F-Droid. It currently has **no held-out family-disjoint metric** (in-sample AUROC 1.0 means nothing), so L5 **keeps it disabled** until that metric exists. That refusal is the design working as intended.

Improvement with more data

The whole system gets better as the corpus grows: more — and more *diverse* — benign apps, especially **real banking apps**, would let the model learn true malice instead of a source shortcut, and would fix the L5 weighting problem described next. More data is not a nice-to-have here; it is *the* lever.

L3 FAILURE POINTS

- **Source confound** (above) — the ceiling on what F-Droid-versus-CICMalDroid can prove.
- **Density floor**: if feature extraction produces too few columns — a packed or broken app — the vector is refused (status: `skipped`) rather than scored on garbage.

L4 · GENAI REASONING (VERIFIED DEOBFUSCATION)

Code: `L4/deobfuscate.py` · `L4/verify.py` · `L4/provider.py` · `L4/scorer.py` · `L4/reasoning_trail.py` · `L4/verify_verdict.py` · `L4/knowledge/`

Job: use a large language model to **explain** the suspicious classes L1 flagged and **decode** obfuscated strings — while guaranteeing the model **cannot lie into the report**. L4 contributes evidence and prose; by default it is **worth zero points** in the score.

Deobfuscation, and why an LLM

Malware hides its intent: renames identifiers to `a`, `b`; encodes its C2 URL as base64. A human analyst reading the decompiled class can often say “this is an SMS interceptor; this base64 string is the C2.” An LLM can do that reading at scale. L4 targets **only the 6–8 classes L1 already**

flagged (their location fields) — not the whole app. Summarising every class of a decompiled app would cost 30–40 minutes of model time and mostly describe framework code.

We use **OpenRouter's free tier** (default model `nvidia/nemotron-3-ultra-550b:free`, with a fallback chain because free endpoints rate-limit). Cost target: **\$0.00 per call**, with a hard budget cap (CostLedger) that refuses a call which would exceed the budget — because a loop bug in a paid API is a bug that *spends money until someone notices*.

The three-agent chain

For each flagged class, L4 runs **three separate LLM calls**, each with deliberately restricted inputs (`deobfuscate.py::explain_class`):

1. **Analyst**. Sees the source, context, and retrieved knowledge. Outputs structured JSON: `purpose`, `renamed identifiers`, `decoded_strings`, `api_calls`, `iocs`, `behaviours`, `confidence`.
2. **Reasoning-trail agent** (`reasoning_trail.py`). Sees **only the claims that survived mechanical verification** — never the raw source. Builds a cited chain: each reasoning step must cite a knowledge-base entry ID or say “no KB citation.”
3. **Adversarial verifier** (`verify_verdict.py`). Sees the source, the trail, and the citations. Its job is to **find a flaw**: is there a benign explanation? Is a citation a stretch or fabricated? Outputs `confirmed` / `weakened` / `refuted`.

Mechanical verification — the anti-hallucination core (`verify.py`)

This is the most important idea in L4. **Every claim a computer can check is checked before it is kept.**

CLAIM	MECHANICAL CHECK
<code>decoded_strings</code>	We re-decode the literal ourselves (<code>base64</code> , <code>base32</code> , <code>hex</code> , even <code>gzip'd</code>) and compare.
<code>renamed</code>	The obfuscated identifier must actually occur in the code (word-boundary match).
<code>api_calls</code>	The API name must actually occur in the code.
<code>iocs</code>	Must already be in L1's extracted IOC list — the model may never introduce a new indicator.
<code>matched_pattern</code>	The cited KB ID must be one the retriever actually returned .
<code>behaviours</code>	Free text — no mechanical check, so it is carried as unverified , never as fact.

A claim that fails is **dropped** — and the drop is recorded for audit — not down-weighted. The worked example that justifies the whole layer: a candidate model confidently decoded `aHR0cDovLzE5Mi4xNjguMS4xMDAvZ2F0ZS5waHA=` as `.../get.php`. The true value is `gate.php`. That is fluent, plausible, exactly the shape of a real finding — and **refuted in a microsecond by one call to `base64.b64decode`**. No amount of AI cleverness catches this; only mechanical re-checking does.

Even the *verifier* isn't trusted on citations: every `kb_id` the trail cites is cross-checked against the real retrieval results in code. A citation that doesn't resolve is a **fabricated citation**, full stop.

The “grounded” idea, and the deterministic score (`scorer.py`)

After verification, a **pure function** — no LLM call — assigns the class a 0–10 score from four inputs: kept claims, dropped claims, KB matches, and the verifier verdict.

- **Grounded** — a KB match survived *and* the verifier confirmed → 5–10.
- **Plausible** — kept mechanical claims but no KB grounding → 3–5.
- **Hunch** — no kept claims, but a suspicion the verifier didn't refute → 1–4.
- **Refuted** → 0.

Because it is deterministic, re-running it on the same inputs gives the same number — auditable, like everything else.

The RAG (retrieval), and “why not a vector DB?”

RAG = retrieval-augmented generation: before the model reasons, we retrieve relevant reference material and hand it over. Our knowledge base (`L4/knowledge/kb.json`) has entries from four sources: **MITRE ATT&CK Mobile** techniques, our own **YARA rule** descriptions, about **28 hand-curated India banking-malware patterns**, and **8 emerging 2024–2026 techniques**.

Retrieval (`retriever.py`) is **TF-IDF plus cosine similarity** — a classic bag-of-words vector method combining word-level and character-n-gram features. Given a class's strings, it returns the top three most similar KB entries.

Why TF-IDF and not a neural embedding plus a vector database (FAISS, Chroma, etc.)?

- The KB is **small** — a few hundred entries. A vector DB earns its keep at millions of vectors; here it would be heavier infrastructure for no measurable gain.
- TF-IDF is **transparent and deterministic** — you can see *why* two texts matched (shared terms), which fits the project's “everything auditable” ethos. A dense embedding match is a black box.

- No extra service to deploy, no model to download — better for **on-prem and air-gapped** banking deployments.
- Our queries are literal code tokens (`sendMessage`, `createFromPdu`), where lexical overlap works extremely well — we even split camelCase so `sendMessage` matches prose that says “send text message.”

So a vector DB is **not** better *for this problem at this scale*. It would become worth it only if the KB grew to tens of thousands of entries.

Crucial scope rule: RAG grounds claims about the **threat landscape** (“this resembles a known technique or family”) — it is **never** allowed to settle a claim about *this sample's bytes*. Those are settled mechanically by `verify.py`. And the RAG **does not learn from apps that pass through it**: `kb.json` only changes when a human re-runs `build_kb.py`; nothing a scan discovers is written back.

L4 FAILURE POINTS

- Free-tier rate limits — mitigated by the fallback chain plus an optional paid proxy.
- The model can still be wrong on *unverifiable* prose (behaviours) — which is exactly why that is carried as unverified, not as fact.

L5 • HYBRID SCORING & THE SMOKING-GUN GATES

Code: `L5/score.py` · `L5/policy.yaml` · `L5/gates.py` · `L5/confidence.py` · `L5/l5.py` · `signals.py`

Job: combine everything into a single **auditable** 0–100 score with a band (Informational / Low / Medium / High / Critical), where **every point traces to a named signal and an evidence ID**.

The design decision that shapes everything

The original proposal suggested a blend like $0.45 \cdot \text{rules} + 0.30 \cdot \text{ml} + 0.25 \cdot \text{llm}$. We **rejected** it: it sums incommensurable units with weights fitted to nothing. Instead L5 is an **additive log-odds** scheme with a versioned policy file, plus **smoking-gun gates**. And a standing rule: **the LLM contributes zero points** — there is literally no code path from L4 into the score by default.

How the score is built — four stages

1. **Accumulate signals.** A “signal” — defined once in `signals.py`, shared so the pricing and spending code cannot drift — is a priceable observation: a YARA rule fired (`yara:<RuleName>`), a brand claim (`10:brand_claim`), the SMS trifecta (`10:sms_trifecta`), a cert anomaly. Each carries the evidence IDs that produced it and each has a **weight** (below). Within a “family”

- all SMS rules, say — weights are **discounted geometrically and capped**, because three SMS rules firing on one class is close to *one* fact, not three; naive addition manufactures false confidence.
- 2. **Map to 0–100** via a sigmoid with two fitted anchors: s_0 (the log-odds at which malware and benign are equally likely — “score 50 *means* a coin-flip”) and a temperature T , set so the 1%-benign-false-positive point lands exactly on score 85, the Critical edge.
- 3. **Apply the ML priors** — L3 (± 10 , cannot reach Critical alone), then L3b (also ± 10 , currently disabled).
- 4. **Apply gates as floors** — $\max(\text{score}, \text{floor})$. A gate can only *raise* a score to a floor, never override or lower it. If the additive score already exceeds the floor, the full reasoning survives.

Where the weights come from — this is the clever part

Weights are **computed, not asserted**, by a separate tool (`tools/rule_firing_report.py`, called “A4”). For each signal it counts how often it fires on **malware** versus **benign** apps and computes a **Jeffreys-smoothed log odds ratio**:

```
weight = log((m+0.5)/(M-m+0.5)) - log((b+0.5)/(B-b+0.5))
```

where m , M = malware hits and total, and b , B = benign hits and total. In English: *a signal that fires much more often on malware than on benign earns a large positive weight; one that fires equally often on both earns roughly zero*. The “+0.5” is Jeffreys smoothing — it keeps the maths sane when a count is zero.

The smoking-gun gates (`gates.py`)

A gate is a specific *combination* that, if present with **distinct evidence**, sets a **Critical floor** of 85. Four are defined:

GATE	COMBINATION	STATUS
G1	Bank impersonation + credential-harvesting UI	Armed
G2	SMS interception + exfiltration to a C2	Armed
G3	Accessibility abuse + overlay on a <i>known bank package</i>	Disabled — no evidence source yet for “which bank”
G4	Dropper permission + embedded second-stage APK	Disabled — L0 doesn't record these yet

Two safety properties:

- **Ternary evaluation.** Each gate leg is `fired` / `not_fired` / **indeterminate**. If a leg's input is missing *because analysis failed* — `jadx` never produced the file — it is indeterminate, **never false**; otherwise a *failed* analysis would read as a *clean* app.
- **Distinct evidence required.** The two legs of a gate must cite *different* finding IDs. Otherwise a single rule that detects “reads SMS and forwards it” would satisfy both halves of G2 by itself — a rule wearing a gate's costume. Disabled gates ship **off** rather than “on but always false”, because “gate didn't fire” would otherwise look like evidence of *innocence*.

The current status — and why it's a feature, not a bug

Here is the honest state, and it is central to the project's credibility. The weights are only trustworthy if the **benign denominator B is large enough**. Early on, with only **4 benign apps**, the maths *inverted*: 32 of 45 signals were priced as evidence of being *benign* — including the brand-claim differentiator at -1.90 . With $B = 4$, the statistical uncertainty (the Jeffreys upper bound on “0 hits out of 4”) is 0.44 — that is, “0/4 false positives” really means “somewhere between 0% and 44%.”

So the system **stamps its own score unsupported** and **refuses to arm the gates** whenever the weights' support is too weak. The closed-form requirement: you need $B \geq 213$ benign apps before the differentiator can even earn a positive weight. That drove a large **benign-corpus effort** — fetching around 600 F-Droid apps — and a re-measurement. The committed calibration (`policy.yaml`) is **fitted** on `n_malware = 640`, `n_benign = 604`.

A system that says “I am not sure enough to score this yet” is more trustworthy than one that always emits a confident number. That refusal is the design working. It is the opposite of VirusTotal's black-box confidence.

Confidence is a *separate* axis

`confidence.py` computes a confidence score that **never touches the score**. A failed detonation, a partial decompile, unsupported weights — these reduce *confidence*, reported next to the score, rather than silently corrupting the score itself. This lets us say “score 88, but confidence low because we couldn't detonate it” — two honest numbers instead of one dishonest one.

Machine-state note

On this specific demo machine the external data partition isn't mounted, so the fitted **weights file, the spines, and the scores aren't present locally** — L5 needs them, or a fresh corpus run, to score. The *code and design* are complete; the *measured artefacts* live on the data drive.

L6 · OUTPUT & UX

Code: L6/report.py · L6/api.py · L6/export.py · L6/web/ · run.py

Job: present the verdict so a human can **check** it, and export machine-readable indicators for defenders.

- **HTML report** (report.py) — one **self-contained** file per sample, with no external CSS, JS, or fonts, so it opens on an air-gapped analyst workstation and prints to PDF. Every number is traceable: each scoring contribution names its signal and evidence ID; each finding shows the rule, file, and matched string. **Uncertainty is displayed, not buried** — confidence sits next to the score, coverage gaps are listed by name, and an unsupported stamp is a banner across the top, not a footnote.
- **FastAPI dashboard / control panel** (api.py, web/) — upload or pick an APK, run the pipeline, watch progress live, view the emulator screen, browse samples and scores. Includes an ai=1 toggle to *compare* the score with and without L4's optional contribution.
- **IOC export** (export.py) — **STIX 2.1, CSV, YARA, and Sigma**. This is the payoff for a defender: the confirmed indicators drop straight into a SIEM, firewall, or threat-intel platform.
- **run.py** — a one-window desktop GUI that runs L0 → L6 on one APK end to end, emulator launch included, for a live demo.

The hackathon scores five criteria. Here is exactly where the project earns each, with concrete pointers.

1 · Novelty – 30 points

- **Impersonation-first detection.** The headline question — “is this *pretending to be a specific Indian bank / UPI / gov app?*” — is not what MobSF, VirusTotal, or generic AV ask. L0's whole-token brand engine plus signer-anomaly correlation is genuinely new framing.
- **Anti-hallucination AI, done right.** L4's three-agent chain with **mechanical verification** — the model explains, but every checkable claim is re-checked by a decoder, parser, or substring search, and fabricated citations are caught in code — is a novel, disciplined way to use an LLM in security without letting it invent evidence (the `gate.php` example).
- **A system that refuses to overclaim.** The unsupported-stamp mechanism (L5 / T24) — self-measuring statistical support and *declining to score* when it is too weak — is unusual and intellectually honest.
- **Evidence spine with stable IDs** — a single auditable record where every downstream claim cites, rather than re-asserts, evidence.

2 · Technical feasibility – 30 points

- **It runs today, end to end.** L0, L1, L3, L4, and L6 execute on a single APK now; L2 detonation is demonstrated — XBot's live C2 capture, 2026-08-26.
- **Real, standard tooling:** androguard, jadx (bundled with its own JDK), YARA, a KVM-accelerated Android emulator, Frida, mitmproxy, DroidBot, LightGBM, FastAPI. Nothing exotic or hand-wavy.
- **Measured, not claimed.** Detection went 8% → 37% after the dex-scan fix; the benign corpus grew to 604; calibration is fitted on 640 + 604. Every number in the docs came from running something.
- **On-prem friendly:** local threat cache, CPU-only ML, self-contained reports, TF-IDF RAG with no external vector DB, and a LocalProvider stub that honestly raises rather than pretending air-gapped LLM inference exists.
- **Verified network isolation** — fail-closed iptables plus a positive/negative proof. You can run *live* malware safely, which many “dynamic analysis” demos cannot actually claim.

3 • Model explainability – 20 points

- **Every point traces to evidence.** `L5/l5.py --explain` prints the full audit trail: each contribution, its weight, the evidence ID behind it, the redundancy discount, and which constraint was binding.
- **Weights are computed from measured counts** (Jeffreys log-odds), not hand-set — and each carries a *support stamp*.
- **Calibrated probabilities** (L3 isotonic calibration) — a “0.9” means about 90%.
- **The LLM's reasoning is a *cited chain***, and its unverifiable prose is clearly marked unverified.
- **Confidence is a separate, visible axis** — the report never hides uncertainty inside the score.

4 • Scalability – 10 points

- **Content-addressed, resumable corpus runner** (`tools/corpus_run.py`): one sample at a time, dedup by SHA-256, resumes on `ruleset_version`, reclaims jadx disk per sample. The full corpus of around 705 samples runs in about 57 minutes with flat disk.
- **The ML model is cheap** — LightGBM, CPU, sparse features — and improves as the corpus grows.
- **Clear scaling levers:** parallel L1 scanning (YARA releases the GIL), content-hash caching to skip re-analysis, and a documented decision about folding in the 2,505-sample CICMalDroid banking set.
- **Honest limits stated** — L2 detonation is inherently the slow, serial step (about one sample at a time in one emulator), and the free-tier LLM rate-limits. Both have named mitigations: snapshots and parallel emulators; a paid fallback proxy.

5 • UI design – 10 points, essentially L6

- **A forensic “case file” aesthetic**, not a build dashboard — the tool's own vocabulary (evidence, findings, fingerprints, confidence bands) is the visual metaphor (`run.py` GUI plus L6/web).
- **Self-contained, air-gap-ready HTML report** that prints to PDF.
- **Traceable by design:** click from a score down to the exact rule, file, and line.
- **Uncertainty surfaced, not buried:** confidence beside the score, gaps listed, unsupported as a banner.
- **Live control panel** with real-time progress, an emulator screenshot, and an AI-on / AI-off comparison toggle.

PART 5 • THE GLOBAL STORY

Read as one system, the pipeline tells a coherent, defensible story.

1. **The problem is fraud, not just insecurity** (Part 1). That reframing is the source of the **novelty**: we ask “is this pretending to be *your bank*?” and answer it with evidence.
2. **Evidence flows one way and accumulates in one place** — the spine — so that by the time we assign a number, every input is on the table with a stable ID. This is the backbone of **explainability**: the score is not a verdict handed down, it is a sum you can audit line by line.
3. **Three independent kinds of evidence** — static patterns (L1), runtime behaviour (L2), statistical resemblance (L3) — plus **AI explanation that cannot lie** (L4). Each is bounded and honest about its own scope: L3 cannot make a banking claim; L4 contributes zero points; the LLM's facts are mechanically re-checked. This layered, cross-checked design is the **technical feasibility** story — real tools, real captures, real numbers.
4. **The scoring refuses to overclaim** (L5). Weights are *measured*, gates need *distinct evidence*, and when the benign denominator is too small the system *stamps itself unsupported* rather than bluffing. This is simultaneously **novel** — few tools do this — and the deepest form of **explainability**: knowing, and saying, *how much you don't know*.
5. **It scales along a clear path** (Part 4.4) and **presents itself honestly** (L6) — every number clickable to its source, uncertainty on the surface.

THE SINGLE SENTENCE THAT CAPTURES THE WHOLE PROJECT

MobSF tells you an app is insecure; VirusTotal tells you 35 engines dislike it. **We tell you *this app is pretending to be your bank, here is the exact evidence, here is how confident we are, and here is what we're still unsure about.***

End of guide. For deeper internals see CLAUDE.md (the working context and the list of hard-won traps) and docs/PROJECT_LOG.md (measurements and rationale).